

given below to create our `config.json`:

```
$ cat > config.json <<EOL
{
    "Args": ["server.js"],    #
    This is the entrypoint for our image.
    "MapDirs": {              #
    Maps a local directory to a path inside
    the image.
        "src/*": "./"
    },
    "RunConfig": {
        "Ports": ["3000"]    #
    Exposes mentioned port
    }
}
EOL
```

Step 2: Build and run our unikernel

Now it's time to build and run our unikernel. As our application is a Node.js application, we will use the `eyberg/node:v16.5.0` package to load it. This is a prebuilt package that Ops will pull from its repository. We could build our own package but that will be a whole new discussion and cannot be included here because of its sheer complexity. To build and run our first unikernel, execute the command given below from within our staging area:

```
$ ops pkg load eyberg/node:v16.5.0 -f -c config.json
```

Once the unikernel is up and running, we should see an output similar to the screenshot in Figure 4.

Open any browser, navigate to `http://localhost:3000` or `http://<Your private IP>:3000`, and start using the calculator which is running in our first Nanos unikernel.

One thing we will notice is that the command captures our shell session. Unfortunately, there is no way to run

Nanos in detached mode. We can add `'&'` at the end of the Ops command to leverage the BASH shell's capability to execute commands in the background. The new command will be as shown below:

```
$ ops pkg load eyberg/node:v16.5.0 -f -c config.json &
```

Deploying unikernels to the cloud (Amazon Web Services)

Let us now consider a real scenario where we deploy our application on the AWS cloud as part of production deployment. Let's assume this is an application that will be used by end users. So far the application is working in our local dev environment. To deploy it to the cloud, we will have to make sure we meet the prerequisites given below:

- An AWS account (free tier account will work)
- AWS CLI v2 installed and configured with an IAM access/secret keys
- An S3 bucket

We will not discuss AWS account creation, AWS CLI installation, and configuration here for obvious reasons. In case we need assistance, there is loads of documentation available over the internet. We can also use AWS CLI to create the bucket. Once that has been created, don't forget to note down the bucket name as we will need it in our next steps. That being said, our next step will be to edit the `config.json` file we created earlier.

Step 1: Editing config.json specifically for AWS deployment

The `config.json` we created in the earlier step served a different purpose. Modification made to the configuration

will now be used specifically to deploy the application to AWS. The commands given below need to be appended to the existing `config.json` file:

```
"CloudConfig" :{
    "Platform": "aws",
    "ProjectID" : "prod-0001",
    "Flavor": "t2.micro",
    "Zone": "us-east-1",
    "BucketName": "nanovm-bucket-
prod-0001"
}
```

For this demo, we have mentioned the bare minimum. If the scenario demands, we can provide VPC details, tags, security groups, and image names. For more details on configuration options, you can go to <https://docs.ops.city/ops/configuration>.

Step 2: Building a unikernel image

Next, we have to build the image that will be used to create the EC2 instance in our AWS account. One important thing to note is that this image will be created on AWS. To be exact, this is an Amazon machine image (AMI) created using the pre-built Ops Node.js package. To create the image, we will be executing the command given below:

```
$ ops image create -t aws -c config.json --package eyberg/node:v16.5.0
```

Once the above command is successfully executed, we will get an output similar to the screenshot given in Figure 5.

Step 3: Create an instance of unikernel

This will be our moment of truth, as in this step we will launch an EC2 instance from the image we created in the previous step. In our `config.json` we could have easily mentioned an image name for our unikernel but didn't do that. So, in our case Ops will provide a default image name based on the package we used to build the image. We can use the command given below

```
shambo@unikernel:/opt/calcul-app$ ops pkg load eyberg/node:v16.5.0 -f -c config.json
100% |████████████████████████████████████████████████████████████████████████████████| [1s:0s]
booting /home/shambo/.ops/images/node ...
en1: assigned 10.0.2.15
en1: assigned FE80::9C21:E3FF:FE5E:20B8
Server running on port 3000
```

Figure 4: Node.js application running on port 3000